Blood Cancer Detection -- Hybrid Ensemble Deep Learning

Comprehensive documentation for the Blood Cancer (ALL) Detection system using a hybrid ensemble of pretrained CNN backbones.

Table of Contents

1. Project Overview
2. System Architecture
3. Installation & Setup
4. Dataset
5. Model Architecture
6. Pipeline Stages
7. Web Application
8. Testing
9. Configuration Reference
10. Troubleshooting
11. API Reference
12. Deployment Notes

---

1. Project Overview

What This Project Does

This project implements an automated blood cancer detection system that classifies microscopic blood cell images into four categories of Acute Lymphoblastic Leukemia (ALL):

Class	Description
Benign	Healthy/normal cells
[Malignant] early Pre-B	Early Pre-B stage ALL
[Malignant] Pre-B	Pre-B stage ALL
[Malignant] Pro-B	Pro-B stage ALL

The system uses a **hybrid ensemble** of three pretrained CNN backbones (MobileNetV2, EfficientNetB0, DenseNet121) combined through weighted soft voting and a stacked logistic regression meta-classifier. It also provides Grad-CAM visual explanations to highlight which regions of the image influenced the prediction.

Target Performance

Metric	Target
Accuracy	>= 96%
Macro F1	>= 95%
Ensemble margin over best single model	>= 0.5 pp
Per-class recall	>= 90%

---

2. System Architecture

High-Level Data Flow

```
Kaggle Dataset (or Demo Data)
        |
        v
   data/raw/ (class-named folders)
        |
        v
  Stratified Split (70/15/15)
        |
   +----+----+
   |    |    |
train  val  test
   |    |    |
   v    |    v
 Three CNN    |   Evaluation
 Backbones    |   (evaluate.py)
   |          |
   v          v
 Ensemble Fitting (on val)
   |
   +-- Weighted Voting (F1-proportional weights)
   +-- Stacked Meta-Classifier (LogisticRegression on 12 features)
   |
   v
 Best ensemble selected (by macro-F1 on test)
   |
```

```
   v
 Prediction API + Web UI + Grad-CAM
```

Module Dependency Graph

```
src/config.py       <-- imported by all modules as C
    |
src/download.py     (dataset acquisition, synthetic data)
src/data.py         (splits, tf.data pipelines, augmentation)
src/models.py       (backbone construction, phase control)
    |
src/train.py        (two-phase fine-tuning, uses data + models)
    |
src/ensemble.py     (voting + stacking, uses data + models)
    |
src/predict.py      (inference, uses ensemble + models + gradcam)
src/evaluate.py     (test metrics + plots, uses ensemble + models + gradcam)
src/gradcam.py      (Grad-CAM heatmaps, uses models)
    |
src/pipeline.py     (stage orchestration)
    |
run.py              (CLI + web server entry point)
api/main.py         (FastAPI backend)
api/jobs.py         (background job runner)
```

---

3. Installation & Setup

Prerequisites

- Python 3.10 or later
- pip
- (Optional) NVIDIA GPU with CUDA for accelerated training
- (Optional) Kaggle account for downloading the real dataset

Environment Setup

```
# Clone the repository
git clone <repo-url>
cd blood-cancer-detection

# Create a virtual environment
python -m venv .venv
source .venv/bin/activate   # Linux/macOS
```

```
.venv\Scripts\activate      # Windows

# Install dependencies
pip install -r requirements.txt
```

Key Dependencies

Package	Version	Purpose
tensorflow	2.15.1 (py<3.12) / 2.21.0 (py>=3.12)	Deep learning framework
keras	3.14.0	High-level neural network API (py>=3.12)
fastapi	0.136.3	Web API framework
uvicorn	0.49.0	ASGI server
scikit-learn	1.7.2	Meta-classifier, metrics
streamlit	1.50.0	Fallback UI
kagglehub	1.0.2 / 0.3.13	Dataset download
matplotlib / seaborn	3.10.7 / 0.13.2	Plotting
opencv-python-headless	4.10.0.84	Image processing / Grad-CAM overlay
pillow	11.3.0	Image I/O

Kaggle Credentials

To download the real dataset, configure Kaggle credentials using one of these methods:

1. **Environment variables:** `bash export KAGGLE_USERNAME=your_username export KAGGLE_KEY=your_api_key`
2. **kaggle.json file** at `~/.kaggle/kaggle.json` : `json {"username": "your_username", "key": "your_api_key"}`
3. **Kaggle Access Token** at `~/.kaggle/access_token` (KGAT token)
4. **Via the Web UI** -- enter credentials or upload a token through the data setup page.

Without credentials, the system falls back to generating synthetic demo data.

---

4. Dataset

Source

The dataset comes from Kaggle and contains microscopic images of blood cells categorized into four ALL subtypes.

Classes

Index	Class Name	Description
0	Benign	Normal/healthy blood cells
1	[Malignant] early Pre-B	Early Pre-B ALL
2	[Malignant] Pre-B	Pre-B ALL
3	[Malignant] Pro-B	Pro-B ALL

Data Splitting

Images are split using stratified sampling:

Split	Fraction	Purpose
Train	70%	Model training
Validation	15%	Ensemble fitting, early stopping, LR scheduling
Test	15%	Final evaluation only (accessed exclusively by `evaluate.py`)

Split metadata is saved to `data/splits/meta.json` and CSV files (`train.csv`, `val.csv`, `test.csv`).

Download Instructions

```
# Download from Kaggle (requires credentials)
python run.py --cli download

# Or generate synthetic demo data (no credentials needed)
python run.py --cli demo-data
```

Demo / Sample Data

- **Demo data** (`data/demo/`): 50 synthetic images per class at 128px. Uses class-specific shapes and colors (circles, squares, triangles, rings) on noisy backgrounds.
- **Sample data** (`data/sample/`): 2 images per class at 64px. Used automatically in smoke-test mode.

The synthetic data generator (`src/download.py:make_synthetic`) creates distinct geometric patterns for each class:

Class	Shape	Color
Benign	Circle	Blue
[Malignant] early Pre-B	Square	Red
[Malignant] Pre-B	Triangle	Green
[Malignant] Pro-B	Ring	Yellow

---

5. Model Architecture

CNN Backbones

Three ImageNet-pretrained backbones are used, each with the classification head replaced:

MobileNetV2

- Lightweight architecture optimized for mobile/edge deployment
- Grad-CAM target layer: `out_relu`

EfficientNetB0

- Compound-scaled architecture balancing depth, width, and resolution
- Grad-CAM target layer: `top_activation`

DenseNet121

- Dense connectivity pattern for feature reuse
- Grad-CAM target layer: `relu`

Classification Head

Each backbone receives the same custom head (3 layers):

```
Base (ImageNet, no top)
  -> GlobalAveragePooling2D
  -> Dropout(0.3)
  -> Dense(4, activation='softmax')
```

The model is built as a **flat** (non-nested) Keras model for compatibility.

Two-Phase Training Strategy

Phase 1 -- Head Training: - All base layers frozen - Learning rate: 1e-3 (Adam) - Duration: 5 epochs - Purpose: train the new classification head on blood cell features

Phase 2 -- Fine-Tuning: - Top 30% of base layers unfrozen (BatchNorm layers remain frozen) - Learning rate: 1e-5 (Adam) - Duration: remaining epochs up to 15 total - Callbacks: EarlyStopping (patience 5), ReduceLROnPlateau (patience 2), ModelCheckpoint (best val_loss) - Class weights: computed via `sklearn.utils.class_weight.compute_class_weight("balanced")`

Ensemble Methods

Weighted Soft Voting

- Each backbone's softmax output is weighted proportional to its validation macro-F1
- Weights saved to `models/ensemble_weights.json`
- Final prediction: `argmax(sum(w_i * p_i))`

Stacked Meta-Classifier

- A multinomial Logistic Regression (`C=1.0` , `max_iter=1000`)
- Input: concatenated softmax outputs from all 3 backbones (12 features)
- Trained on validation set predictions
- Saved to `models/meta_clf.pkl` via joblib

Ensemble Selection

- `evaluate.py` tests both methods on the held-out test set
- Selects the method with higher macro-F1 (ties go to voting)
- If the best ensemble does not beat the best single model by the margin target (0.5 pp), fallback strategies are tried: kappa-weighted, F1-squared, drop-one-backbone

Grad-CAM

Implements Selvaraju et al. (2017) for visual explanations:

1. Forward pass through the model to a target convolutional layer
2. Compute gradients of the predicted class score w.r.t. that layer's activations
3. Global average pool the gradients to get channel importance weights
4. Weighted combination + ReLU to produce a heatmap
5. Overlay on the original image with JET colormap (alpha=0.4)

---

6. Pipeline Stages

The pipeline is managed by `src/pipeline.py` and can be run via CLI or the web UI.

Available Stages

Stage	Command	Description
`download`	`python run.py --cli download`	Download Kaggle dataset to `data/raw/`
`demo-data`	`python run.py --cli demo-data`	Generate synthetic demo images
`splits`	`python run.py --cli splits`	Create stratified 70/15/15 splits
`train`	`python run.py --cli train`	Two-phase fine-tuning of all backbones
`ensemble`	`python run.py --cli ensemble`	Fit voting weights + stacking meta-classifier
`evaluate`	`python run.py --cli evaluate`	Run test evaluation, generate plots and metrics
`all`	`python run.py --cli all`	Run full pipeline end-to-end

Stage: download

- Uses `kagglehub.dataset_download` to fetch the dataset
- Copies class-named folders into `data/raw/`
- Falls back to demo-data if no Kaggle credentials are found

Stage: splits

- Discovers all images in the data directory via `list_images()`
- Performs stratified train/val/test splitting
- Writes CSV files and metadata to `data/splits/`
- Generates `results/split_counts.csv`

Stage: train

- Builds each backbone with `build_backbone()`
- Phase 1: frozen base, high LR, HEAD_EPOCHS
- Phase 2: partial unfreeze, low LR, remaining epochs
- Saves best checkpoint to `models/<name>.keras`
- Logs training history to `results/history/<name>.csv`

Stage: ensemble

- Loads all trained `.keras` models
- Runs inference on the validation split
- Fits weighted voting (F1-proportional weights)

- Fits stacking meta-classifier (LogisticRegression)
- Saves artifacts to `models/`

Stage: evaluate

- Loads the test split (the **only** module that accesses `test.csv`)
- Computes metrics for each backbone and both ensemble methods
- Selects the best ensemble by macro-F1
- Generates all result files and figures:
- `results/metrics.json` -- all computed metrics
- `results/comparison_table.md` -- Markdown comparison
- `results/NOTES.md` -- target achievement notes
- `results/run_info.json` -- full provenance
- `results/figures/` -- confusion matrices, ROC curves, training curves, comparison bars, Grad-CAM grids

---

7. Web Application

Starting the Web App

```
# Default: FastAPI at localhost:8000 (auto-opens browser)
python run.py

# Custom host/port
python run.py --host 0.0.0.0 --port 9000

# Without auto-opening browser
python run.py --no-browser

# Streamlit fallback UI
python run.py --streamlit
```

FastAPI Backend (`api/main.py`)

The backend serves both the JSON API and the static frontend from the `frontend/` directory.

Authentication

Demo login only (not production-grade): - Username: `admin` - Password: `admin` - Endpoint: `POST /api/login`

Frontend Views

The `frontend/` directory contains a static HTML/CSS/JS web UI with pages for:

- **Login** -- Demo authentication
- **Data Setup** -- Kaggle credential entry, dataset download, demo data generation, split creation
- **Training** -- Backbone selection, training launch with real-time progress via SSE
- **Ensemble** -- Fit ensemble methods
- **Evaluation** -- Run evaluation, view metrics, confusion matrices, ROC curves
- **Prediction** -- Upload an image, see prediction with Grad-CAM overlay
- **Results** -- Browse all generated figures and download result files
- **Settings** -- Switch between ensemble and single-model prediction modes

Background Jobs

Long-running operations (download, train, ensemble, evaluate) run as background jobs via `api/jobs.py`:

- One job at a time (attempting to start a second returns 409 Conflict)
- Each job has a unique 12-character hex ID
- Progress tracked via a callback protocol (`progress_cb(stage, epoch, logs)`)
- Real-time updates available via SSE at `GET /api/jobs/{job_id}/stream`
- Job log limited to 500 lines

---

8. Testing

Running Tests

```
# Run all tests
pytest

# Run with verbose output
pytest -v

# Run a specific test file
pytest tests/test_config.py
```

Test Coverage

The `tests/` directory contains pytest tests covering:

- Configuration loading and parameter validation
- Data splitting correctness and pipeline construction

- Model building and phase transitions
- Training loop execution (smoke mode)
- Ensemble fitting and prediction
- Grad-CAM heatmap generation
- API endpoint responses
- End-to-end pipeline orchestration

Smoke Mode

For fast iteration, use `--smoke` to run with minimal resources:

```
python run.py --smoke --cli all
```

Smoke mode overrides: - Image size: 96px (instead of 224) - Batch size: 4 (instead of 32) - Epochs: 1 per phase (instead of 5+10) - No ImageNet weights - Eager execution enabled - Uses `data/sample/` (2 images/class)

---

9. Configuration Reference

All configuration lives in `src/config.py`. The module is typically imported as `C`.

Image & Data

Parameter	Default	Description
`IMG_SIZE`	224	Input image dimension (square)
`SEED`	42	Global random seed
`SPLIT_FRACTIONS`	`{train: 0.70, val: 0.15, test: 0.15}`	Data split ratios

Training Hyperparameters

Parameter	Default	Description
`BATCH_SIZE`	32	Training batch size
`EPOCHS`	15	Total training epochs (both phases)
`HEAD_EPOCHS`	5	Phase 1 (head-only) epochs
`LR_HEAD`	1e-3	Phase 1 learning rate
`LR_FINE`	1e-5	Phase 2 learning rate

Parameter	Default	Description
`UNFREEZE_FRACTION`	0.30	Fraction of base layers to unfreeze in Phase 2
`DROPOUT`	0.3	Dropout rate in the classification head

Callbacks

Parameter	Default	Description
Early stopping patience	5	Epochs without val_loss improvement before stopping
LR plateau patience	2	Epochs before reducing LR on plateau

Augmentation

Parameter	Default	Description
Rotation	20 degrees	Random rotation range
Zoom	10%	Random zoom range
Brightness	10%	Random brightness adjustment
Contrast	10%	Random contrast adjustment

Ensemble

Parameter	Default	Description
`META_C`	1.0	Logistic regression regularization strength
`META_MAX_ITER`	1000	Max iterations for meta-classifier convergence
`ENSEMBLES`	`["voting", "stacking"]`	Available ensemble methods

Backbones

Parameter	Default	Description
`BACKBONES`	`["mobilenetv2", "efficientnetb0", "densenet121"]`	Available CNN backbones
`ACTIVE_MODES`	`["ensemble", "mobilenetv2"]`	Prediction modes

Smoke Mode Overrides

Parameter	Smoke Value	Description
`SMOKE_IMG_SIZE`	96	Reduced image size

Parameter	Smoke Value	Description
`SMOKE_BATCH_SIZE`	4	Reduced batch size
`SMOKE_EPOCHS_PER_PHASE`	1	Single epoch per phase

Target Thresholds

Metric	Threshold	Description
Accuracy	>= 0.96	Minimum acceptable accuracy
Macro F1	>= 0.95	Minimum acceptable macro-F1
Margin	>= 0.5 pp	Ensemble must exceed best single model by this
Class recall	>= 0.90	Minimum per-class recall

Environment Variables

Variable	Purpose
`BCE_ROOT`	Override the project root directory for all artifact paths
`KAGGLE_USERNAME`	Kaggle API username
`KAGGLE_KEY`	Kaggle API key

---

10. Troubleshooting

Common Issues

Problem: "No Kaggle credentials found" - Ensure `KAGGLE_USERNAME` and `KAGGLE_KEY` are set, or `~/.kaggle/kaggle.json` exists - Alternatively, use `python run.py --cli demo-data` to generate synthetic data

Problem: Out of memory during training - Reduce `BATCH_SIZE` in `src/config.py` - Use `--smoke` mode for testing - Ensure you are not running multiple training sessions simultaneously

Problem: "A job is already running" (409 error) - Only one background job can run at a time - Wait for the current job to finish or restart the server

Problem: Low accuracy on demo data - Demo data uses simple synthetic shapes, not real blood cell images - Results on demo data are not representative of real-world performance - Download the Kaggle dataset for meaningful results

Problem: TensorFlow not detecting GPU - Verify CUDA and cuDNN installation: `python -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"` - Ensure compatible CUDA version for your TensorFlow version

Problem: Import errors or missing modules - Ensure all dependencies are installed: `pip install -r requirements.txt` - Check Python version compatibility (3.10+)

Problem: Test split access error - The test split is intentionally restricted; only `evaluate.py` can access it - `load_split("test")` raises an error by design to prevent data leakage

Problem: Models not found for prediction - Run the full pipeline first: `python run.py --cli all` - Or fetch pretrained models: `python run.py --fetch-models <URL>`

Problem: Smoke mode uses wrong data directory - If real-data models already exist, smoke mode redirects to `.smoke/` via `BCE_ROOT` - This prevents overwriting production models with smoke-test artifacts

---

11. API Reference

Base URL: `http://localhost:8000`

Authentication

POST /api/login

Demo login endpoint.

Request:

```
{"username": "admin", "password": "admin"}
```

Response (200):

```
{"ok": true}
```

Response (401):

```
{"detail": "Invalid credentials"}
```

System Status

GET /api/status

Returns full system status including splits, models, ensemble state, metrics, active job, and configuration.

Response (200):

```
{
  "splits_ready": true,
  "models": {"mobilenetv2": true, "efficientnetb0": true, "densenet121": true},
  "ensemble_ready": true,
  "metrics": {"accuracy": 0.97, "macro_f1": 0.96},
  "active_job": null,
  "config": {"img_size": 224, "batch_size": 32, "epochs": 15}
}
```

GET /api/health

Lightweight health check.

Response (200):

```
{"status": "ok"}
```

Data Management

POST /api/data/kaggle-creds

Save Kaggle username and API key.

Request:

```
{"username": "your_username", "key": "your_api_key"}
```

POST /api/data/kaggle-token

Save a Kaggle access token (KGAT).

Request:

```
{"token": "KGAT..."}
```

POST /api/data/download

Start async Kaggle dataset download.

Response (202):

```
{"job_id": "a1b2c3d4e5f6"}
```

POST /api/data/demo

Start async demo data generation.

Request (optional):

```
{"smoke": false}
```

POST /api/data/splits

Synchronously create train/val/test splits.

GET /api/data/samples

Get sample images for UI preview.

GET /api/data/test-images

Get exported test images for prediction demo.

Training

POST /api/train

Start async training of specified backbones.

Request:

```
{
  "models": ["mobilenetv2", "efficientnetb0", "densenet121"],
  "smoke": false,
  "epochs": 15
}
```

Response (202):

```
{"job_id": "a1b2c3d4e5f6"}
```

Ensemble

POST /api/ensemble

Start async ensemble fitting.

Response (202):

```
{"job_id": "a1b2c3d4e5f6"}
```

Evaluation

POST /api/evaluate

Start async evaluation on test set.

Response (202):

```
{"job_id": "a1b2c3d4e5f6"}
```

Full Pipeline

POST /api/pipeline/all

Run the complete pipeline (download/demo -> splits -> train -> ensemble -> evaluate).

Request (optional):

```
{"smoke": false}
```

Jobs

GET /api/jobs/{job_id}

Get job status and progress.

Response (200):

```
{
  "id": "a1b2c3d4e5f6",
  "kind": "train",
  "state": "running",
  "stage": "mobilenetv2",
```

```
  "progress": 45.2,
  "logs": ["Phase 1 epoch 1/5 - loss: 0.82 - val_loss: 0.65"],
  "error": null
}
```

GET /api/jobs/{job_id}/stream

Server-Sent Events stream for real-time job progress updates.

Results

GET /api/results

Full results payload with metrics, per-class performance, confusion matrices, figure URLs, training histories, and notes.

GET /api/results/files/{name}

Download a result file. Valid names: `metrics.json`, `comparison_table.md`, `NOTES.md`.

Prediction

POST /api/predict

Upload an image for classification.

Request: multipart/form-data with an `image` file field.

Response (200):

```
{
  "class": "Benign",
  "class_idx": 0,
  "confidence": 0.987,
  "probs": [0.987, 0.005, 0.006, 0.002],
  "per_model_probs": {
    "mobilenetv2": [0.99, 0.003, 0.005, 0.002],
    "efficientnetb0": [0.98, 0.007, 0.008, 0.005],
    "densenet121": [0.99, 0.004, 0.004, 0.002]
  },
  "votes": {
    "mobilenetv2": "Benign",
    "efficientnetb0": "Benign",
    "densenet121": "Benign"
  },
  "mode": "voting",
```

```
  "mode_label": "Weighted Voting Ensemble",
  "gradcam": "..."
}
```

Settings

GET /api/settings

Get the active prediction mode.

POST /api/settings

Set the active prediction mode.

Request:

```
{"active_mode": "ensemble"}
```

Static Assets

GET /media/{source}/{rel}

Serve images from sample/demo/kaggle/test directories.

GET /figures/{name}

Serve generated plot PNGs from `results/figures/`.

GET /api/about

Returns paper mapping table and architecture information.

---

12. Deployment Notes

Local Development

```
python run.py                    # FastAPI on localhost:8000
python run.py --streamlit        # Streamlit fallback UI
python run.py --smoke --cli all  # Quick smoke test
```

Production Considerations

- The demo login (`admin/admin`) is **not production-grade**. Implement proper authentication before deploying publicly.
- The job runner is in-process and single-threaded. For production, consider a task queue (Celery, RQ).
- Model files (`.keras`) can be large. Use `--fetch-models <URL>` to download pretrained models from a release.

Running on Google Colab

Use the provided `notebooks/train_colab.ipynb` for training on free GPU: 1. Upload the notebook to Colab 2. Set Kaggle credentials in Colab secrets 3. Run all cells to train and evaluate

Docker (example)

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["python", "run.py", "--host", "0.0.0.0", "--no-browser"]
```

Model Verification

```
# Verify model checksums against MODELS_SHA256.txt
python run.py --check
```

Environment Variable Reference

Variable	Purpose
`BCE_ROOT`	Override project root for all paths
`KAGGLE_USERNAME`	Kaggle API username
`KAGGLE_KEY`	Kaggle API key

Directory Structure (Runtime Artifacts)

```
blood-cancer-detection/
  data/
    raw/              # Downloaded Kaggle images
    demo/             # Synthetic demo images
    sample/           # Minimal smoke-test images
    splits/           # train.csv, val.csv, test.csv, meta.json
```

```
models/
  mobilenetv2.keras
  efficientnetb0.keras
  densenet121.keras
  ensemble_weights.json
  meta_clf.pkl
  active_mode.txt
  ensemble_mode.txt
  ood_stats.json
results/
  metrics.json
  comparison_table.md
  NOTES.md
  run_info.json
  split_counts.csv
  history/          # Per-backbone training CSVs
  figures/          # All generated plots
```